Chapter 1. n8n 기초

> ⏱️ **배정 시간 55분** — 개념 복습 10분 · 설치 확인 15분 · 화면 구성 10분 · 실습 1-1 20분

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] 단순 자동화, LLM Chain, AI Agent의 차이를 설명할 수 있다
- [ ] 로컬 PC에 n8n을 설치하고 `localhost:5678`로 접속할 수 있다
- [ ] 설치나 접속이 안 될 때 스스로 원인을 찾아 해결할 수 있다
- [ ] n8n 화면 구조(워크플로우 목록 → Canvas)와 편집 화면의 4가지 구성 요소를 구분할 수 있다
- [ ] Expression 문법(`{{ $json.필드명 }}`)으로 이전 노드의 데이터를 가져올 수 있다
- [ ] 데이터가 "아이템 단위"로 흐른다는 개념을 이해하고 설명할 수 있다
- [ ] Manual Trigger, HTTP Request, Edit Fields 노드를 연결해 하나의 워크플로우를 완성할 수 있다

📖 이 장의 용어

본문에 나오는 용어입니다. 낯선 용어가 나오면 이 표로 돌아와 확인합니다.

용어	영문 · 약어 풀이	설명
n8n	엔에이트엔 ("nodemation"의 줄임말)	노드를 연결해 자동화 흐름을 만드는 오픈소스 워크플로 도구. 내 PC에 설치해 무료로 사용 가능
노코드	No-code	코드를 직접 작성하지 않고 화면에서 부품을 조립해 프로그램을 만드는 방식
워크플로우	Workflow, 작업 흐름	여러 노드를 순서대로 연결한 하나의 자동화 단위. n8n에서 저장·실행·공유하는 기본 단위
노드	Node	워크플로우를 이루는 작업 단위 하나. HTTP 요청, 조건 판단, 메일 발송 등 기능마다 노드가 있음
Canvas	캔버스, 작업 공간	노드를 배치하고 선으로 연결하는 편집 화면
Trigger	트리거, 시작 조건	워크플로우를 시작시키는 첫 노드. 버튼(Manual), 시간(Schedule), 외부 요청(Webhook), 채팅(Chat) 등

용어	영문 · 약어 풀이	설명
Execution	실행 (기록)	워크플로우가 한 번 돌아간 기록. Executions 탭에서 성공·실패와 각 노드의 데이터를 확인
Expression	표현식	`{{ }}` 안에 적어 앞 노드의 값을 가져오거나 계산하는 문법. fx 아이콘으로 모드 전환 필요
`$json`	—	바로 앞 노드에서 넘어온 현재 아이템의 데이터를 가리키는 변수
`$('노드이름')`	—	바로 앞이 아닌 특정 노드의 출력을 가져오는 문법
`$now`	—	현재 시각을 담은 내장 변수. `.setZone('Asia/Seoul').toFormat('MM/dd')`는 n8n이 내장한 날짜 라이브러리(Luxon)의 문법
JSON	JavaScript Object Notation	`{"이름": 값}` 형태의 데이터 표기 형식. 노드 사이를 오가는 데이터와 API 응답이 모두 JSON
아이템	Item	n8n이 데이터를 다루는 단위. 엑셀의 한 행과 비슷하며, 다음 노드는 아이템마다 한 번씩 실행
API	Application Programming Interface	프로그램끼리 데이터를 주고받는 약속. HTTP Request 노드가 이를 호출
HTTP Request	—	URL로 요청을 보내고 응답을 받는 노드. GET(조회), POST(전송) 등 Method 선택
쿼리 파라미터	Query Parameter	URL의 `?` 뒤에 `이름=값&이름=값`로 붙여 요청 조건을 전달하는 값
위도 / 경도	Latitude / Longitude	지구 위 위치 좌표. 위도는 남북, 경도는 동서. 서울은 약 37.57, 126.98
Edit Fields (Set)	—	필드(이름=값)를 새로 만들거나 고치는 노드. 예전 이름이 Set
Node.js / npm / npx	Node Package Manager / Node Package Execute	Node.js는 JavaScript를 PC에서 실행하는 환경, npm은 패키지 설치 도구, npx는 설치 없이 바로 실행하는 명령. n8n이 Node.js 위에서 동작
localhost:5678	로컬호스트 + 포트	"내 컴퓨터"의 5678번 포트(Port, 프로그램 구분 번호)에서 돌고 있는 n8n에 접속하는 주소
LLM Chain	—	사람이 정한 순서 안에서 LLM이 내용만 처리하는 구조. n8n의 Basic LLM Chain 노드

1.1 AI Agent란? (1일차 복습)

> 1일차 Chapter 1에서 LLM과 Agent의 차이, Tool·Context의 역할을 배웠습니다. 여기서는 그 내용을 n8n 관점에서 한 번 더 정리합니다. 특히 아래 3단 구분은 Chapter 7에서 "무엇을 Agent로 만들고 무엇은 만들지 말아야 하는가"를 판단하는 기준이 됩니다.

1.1.1 단순 자동화 vs LLM Chain vs Agent

"자동화", "LLM Chain", "Agent"는 자주 섞여 쓰이는 말입니다. 이 세 가지를 가르는 기준은 하나, **누가 다음 행동을 정하는가**입니다.

① 단순 자동화 (Automation) — 사람이 정한다

정해진 규칙(if-then)에 따라 정해진 작업을 수행합니다. 판단이 없고, 입력이 같으면 언제나 같은 결과가 나옵니다.

> 예시: "매일 오전 8시에 날씨와 환율을 조회해서, 환율이 기준을 넘으면 경고 문구를 붙여 메일로 보낸다." (Chapter 2에서 만듭니다)

② LLM Chain — 사람이 순서를 정하고, AI는 내용만 처리한다

LLM을 처리 단계 중 하나로 사용하지만, 흐름은 사람이 미리 짜둔 순서대로 고정되어 있습니다. LLM은 "이해"하지만 "다음에 무엇을 할지"는 정하지 않습니다.

> 예시: "회의록을 AI에게 보내 요약 → 액션 아이템 추출 → 공지 메일 초안 작성" (순서 고정, Chapter 3)

③ AI Agent — AI가 정한다

목표만 주어지고, 그 목표를 달성하기 위해 **어떤 도구를 언제 쓸지 LLM이 스스로 판단**합니다. 같은 목표라도 상황에 따라 실행 경로가 달라집니다.

> 예시: "도쿄 출장 준비를 도와줘"라고 하면, Agent가 스스로 날씨 조회·환율 조회·공휴일 조회 도구를 골라 호출하고 결과를 합쳐 답합니다. (Chapter 5)

📊 비교표

구분	단순 자동화	LLM Chain	AI Agent
판단 주체	사람(사전 규칙)	사람(순서 설계) + LLM(내용 처리)	LLM(도구 선택까지)
실행 경로	고정	고정	상황에 따라 가변
예측 가능성	매우 높음	높음	상대적으로 낮음(대신 유연함)
n8n에서 쓰는 노드	IF, Switch, Edit Fields 등	Basic LLM Chain	AI Agent 노드

구분	단순 자동화	LLM Chain	AI Agent
1일차 대응	—	Tool 없는 Agent (Ch2)	Tool을 가진 Agent (Ch3~7)

> 셋 중 무엇이 더 좋다는 뜻은 아닙니다. 결과가 항상 같아야 하는 업무에는 단순 자동화가 정답이고, 예측 불가능한 요청을 받아내야 하는 업무에는 Agent가 맞습니다. 이 과정에서는 Chapter 1~3에서 앞의 두 가지를, Chapter 4부터 Agent를 다룹니다.

✅ 체크포인트 질문

- "쇼핑몰 재고가 10개 이하면 슬랙 알림을 보낸다"는 셋 중 무엇일까요? → **단순 자동화**
- "고객 문의를 보고 환불인지 배송문의인지 스스로 판단해서 다른 처리를 한다"는? → **AI Agent**
- 1일차에 만든 `my-adk`의 Agent는 n8n의 어느 노드에 해당할까요? → **AI Agent 노드** (Tool을 가지고 스스로 선택하므로)

1.2 n8n 설치

n8n은 노드를 연결해 자동화 흐름을 만드는 오픈소스 워크플로 도구입니다. 이 과정은 **로컬 PC에 n8n을 직접 설치**해서 진행합니다. 클라우드 가입이나 결제 없이, 내 컴퓨터에서 n8n을 실행하고 브라우저로 접속해서 사용하는 방식입니다.

1.2.1 ① Node.js 설치

n8n은 Node.js 위에서 동작하므로 먼저 Node.js가 필요합니다. 1일차 Chapter 7에서 설치했다면 이 단계는 건너뜁니다.

1. nodejs.org 접속
2. **Node.js 24** 버전 다운로드 (현재 배포되는 n8n 2.x는 Node.js 24 이상을 요구합니다. Node.js 22에서는 n8n 설치 단계에서 오류가 발생합니다. 다운로드 화면에 표시된 버전이 24가 아니면 버전 선택 메뉴에서 v24를 선택하세요)
3. 다운로드된 설치 파일을 실행하고, 옵션은 모두 기본값으로 두고 **Next**를 눌러 설치 완료

설치 확인

터미널을 엽니다.

OS	여는 방법
Windows	시작 메뉴에서 `cmd` 검색 → **명령 프롬프트** 실행

OS	여는 방법
macOS (참고, 집에서 재현 시)	`Cmd + Space` → `터미널` 검색 후 실행

아래 두 명령어를 차례로 입력합니다.

```
node -v
npm -v
```

`v24.11.0`, `11.6.0`처럼 **버전 숫자가 출력되면 정상**입니다. `node -v` 결과의 첫 숫자가 **24**인지 확인합니다. 뒤의 숫자는 달라도 됩니다.

> 💡 `'node'은(는) 내부 또는 외부 명령이 아닙니다` 같은 오류가 나온다면, 설치 직후라 터미널이 변경 사항을 아직 모르는 상태입니다. **터미널 창을 완전히 닫고 새로 열어서** 다시 입력해 보세요. 대부분 이걸로 해결됩니다.

1.2.2 ② n8n 실행

명령 프롬프트에 아래 한 줄을 입력합니다. 이 과정의 명령은 모두 Windows 명령 프롬프트(cmd) 기준입니다. PowerShell에서는 보안 정책 때문에 `npx`가 차단될 수 있으므로 사용하지 않습니다.

```
npx n8n@2
```

> ⚠️ `@2`를 반드시 붙입니다. n8n 3.0부터는 npx 방식 설치가 지원되지 않으므로, 2.x 버전을 지정해야 이 교재의 절차가 그대로 동작합니다.

실행하면 이런 순서로 진행됩니다.

1. `Need to install the following packages: n8n@2...` 같은 문구와 함께 설치 여부를 물어봅니다 → `y`**를 입력하고 Enter**
2. 처음 실행이라면 다운로드가 시작됩니다. 용량이 커서 **3~10분 정도 걸릴 수 있습니다.** 화면이 멈춘 것처럼 보여도 정상이니 기다려주세요. (실습실 PC는 미리 받아두어 바로 실행됩니다)
3. 여러 줄의 로그가 지나가고, 마지막에 아래 메시지가 나타납니다.

`Editor is now accessible via: http://localhost:5678`

이 메시지가 보이면 성공입니다.

> 💡 처음 실행할 때 **Windows 보안 경고**(Node.js JavaScript Runtime의 네트워크 액세스 허용) 창이 나타날 수 있습니다. 이 과정은 내 PC에서만 접속하므로 **취소**를 눌러도

> `localhost:5678` 접속에는 영향이 없습니다. 실습실 정책에 따라 강사가 안내합니다.

> ⚠️ **터미널 창을 닫지 마세요.** 이 터미널이 곧 n8n 서버입니다. 창을 닫거나 `Ctrl + C`를 누르면 n8n이 즉시 종료되고 `localhost:5678` 접속이 끊깁니다. 실습이 끝날 때까지 켜둔 채로 두고, 다른 명령어를 입력해야 한다면 **터미널 창을 새로 하나 더 여세요.**

1.2.3 ③ 브라우저에서 접속하고 계정 만들기

1. 웹 브라우저(Chrome 권장)를 열고 주소창에 `http://localhost:5678` 입력
2. **Set up owner account** 화면이 나타납니다. 이메일과 비밀번호를 입력합니다.

입력 항목	안내
Email	실제로 받아볼 수 있는 주소가 아니어도 됩니다. 인증 메일이 오지 않습니다.
First / Last Name	아무거나 입력
Password	8자 이상이며 숫자와 대문자를 각각 1개 이상 포함해야 합니다 (화면의 조건 안내를 따르세요)

> 이 계정 정보는 **내 PC 안에만 저장**되며 외부로 전송되지 않습니다. 다만 비밀번호는 복호화가 불가능한 형태로 저장되어 **잊어버리면 찾을 수 없으니** 메모해두세요.

3. 이후 나오는 설문 화면은 건너뛰거나(**Skip**) 대충 응답해도 무방합니다.
4. 워크플로우 목록 화면이 나타나면 준비 완료입니다.

> ⚠️ **실습실 PC에서 계정 생성 화면 대신 로그인(Sign in) 화면이 뜬다면** 이전 수강생의 데이터가 남아 있는 것입니다. 강사에게 알리거나, 아래 1.2.4의 **데이터 초기화** 절차를 따르세요.

설치 확인 체크리스트

- [ ] `node -v`, `npm -v` 버전 숫자 출력 확인
- [ ] `npx n8n@2` 실행 후 `Editor is now accessible via:` 메시지 확인
- [ ] `http://localhost:5678` 접속 성공
- [ ] 계정 생성 완료 후 워크플로우 목록 화면 진입

> 📌 n8n은 영문 UI가 기본입니다. 이 교재의 모든 설명은 영문 UI 기준입니다.

1.2.4 설치 트러블슈팅

증상	원인	해결
계정 생성 화면 대신 **로그인 (Sign in) 화면**이 뜬다	이전에 n8n을 실행한 적이 있어 계정 데이터가 남아 있음	아래 **데이터 초기화** 참고
`Port 5678 is already in use` 오류	n8n이 다른 터미널 창에서 이미 실행 중	열려 있는 다른 터미널 창을 확인하거나 아래 **포트 확인** 참고
`node`, `npm` 명령을 못 찾음	설치 후 터미널이 갱신되지 않음	터미널을 완전히 닫고 새로 열기
`Your Node.js version ... is currently not supported`	Node.js 버전이 지원 범위 밖	Node.js 24 설치
설치 중 `gyp ERR!`, `isolated-vm` 오류	Node.js 버전이 24가 아니어서 미리 빌드된 모듈을 쓰지 못하고 직접 컴파일을 시도함	`node -v`로 버전 확인 후 Node.js 24 설치
다운로드가 오래 멈춰 있음	사내망 프록시·백신이 npm 접속을 차단	네트워크 담당자에게 npm 레지스트리 허용 요청, 또는 개인 네트워크로 전환
접속은 되는데 화면이 계속 로딩 중	브라우저 캐시 문제	시크릿 창(`Ctrl + Shift + N`)으로 재접속

포트 확인

명령 프롬프트에서 실행합니다.

```
netstat -ano | findstr :5678
```

아무것도 출력되지 않으면 n8n이 실행 중이지 않은 상태입니다. `LISTENING` 줄이 출력되면 맨 끝 열의 숫자(PID, 프로세스 번호)를 확인하고 `taskkill /F /PID 번호`로 종료할 수 있습니다.

> 참고(macOS에서 재현할 때): `lsof -i :5678`

데이터 초기화 (계정을 새로 만들고 싶을 때)

n8n의 모든 데이터는 사용자 폴더 안에 저장됩니다. 이 폴더를 지우면 처음 설치한 상태로 돌아갑니다.

먼저 n8n을 **완전히 종료**합니다. 실행 중인 터미널에서 `Ctrl + C`를 누르고(`일괄 작업을 끝내시겠습니까 (Y/N)?`가 표시되면 `Y` 입력), 위의 포트 확인 명령으로 아무것도 안 잡히는지 확인하세요. 프로세스가 살아 있는 채로 폴더를 지우면 데이터가 다시 살아납니다.

명령 프롬프트에서 아래 세 줄을 차례로 실행합니다. 첫 줄은 실행 중인 Node.js 프로그램을 모두 종료하므로, 다른 Node.js 프로그램을 쓰고 있다면 먼저 저장합니다. `실행 중인 프로세스를 찾을 수 없습니다`가 표시되면 이미 종료된 상태이므로 그대로 진행합니다.

```
taskkill /F /IM node.exe
rmdir /s /q "%USERPROFILE%\.n8n"
dir "%USERPROFILE%\.n8n"
```

> 참고(macOS에서 재현할 때): `pkill -f n8n` → `rm -rf ~/.n8n` → `ls ~/.n8n`

마지막 확인 명령에서 **"파일을 찾을 수 없습니다"**가 나와야 정상입니다. 이후 `npx n8n@2`를 다시 실행하고 시크릿 창으로 접속하면 계정 생성 화면이 나타납니다.

> 워크플로우는 남기고 계정만 초기화하려면 `npx n8n@2 user-management:reset`을 사용합니다.

1.3 화면 구성

계정을 만들고 접속하면 곧바로 편집 화면이 열리는 것이 아니라 **워크플로우 목록 화면**이 먼저 나타납니다. 여기서 워크플로우를 하나 열어야 실제 작업 공간(Canvas)으로 들어갑니다.

1.3.1 워크플로우 목록 화면

n8n은 워크플로우를 **Project** 단위로 묶어 관리합니다. 접속하면 기본적으로 `Personal` 프로젝트 안의 워크플로우 목록이 보입니다.

요소	위치	역할
좌측 사이드바	화면 왼쪽	Projects 목록, Overview, Credentials, Data tables, Templates 등 메뉴
워크플로우 목록	화면 중앙	저장된 워크플로우들이 목록으로 표시
Create Workflow 버튼	화면 우측 상단	새 워크플로우 생성

이 과정에서는 Project 개념을 깊이 다루지 않고 기본 `Personal` 프로젝트만 사용합니다. 사이드바의 **Data tables**는 Chapter 6에서 사용합니다.

1.3.2 편집 화면 (Canvas)

목록에서 워크플로우를 클릭하거나 새로 만들면 편집 화면으로 이동합니다. 이 화면은 크게 4개 영역으로 되어 있습니다.

영역	위치	역할
Canvas	화면 중앙	노드를 배치하고 연결선으로 흐름을 만드는 작업 공간
Node Panel	`+` 버튼 클릭 시 나타나는 패널	사용할 노드를 검색하고 추가
Node 설정창	노드 더블클릭 시 팝업	노드의 파라미터 입력 (INPUT / OUTPUT 데이터 미리보기 포함)
Execution 로그	Canvas 상단의 Editor / Executions 탭	실행 이력, 노드별 성공/실패와 데이터 확인

노드(Node)란

- n8n의 모든 작업 단위는 "노드"입니다. **하나의 노드 = 하나의 기능** (HTTP 요청 보내기, 조건 확인하기, 이메일 보내기)
- 노드는 왼쪽에서 오른쪽으로 연결선을 통해 데이터를 주고받습니다.
- 노드 위에 마우스를 올리면 나타나는 ▶ **버튼**을 누르면 **그 노드까지만** 실행해서 결과를 바로 볼 수 있습니다. 이 기능은 실습 내내 쓰게 되니 위치를 꼭 기억해두세요.
- **노드 이름 바꾸기**: 노드를 더블클릭해 설정창을 연 뒤 상단의 이름을 클릭하면 바꿀 수 있습니다. Chapter 2부터 "다른 노드의 값 가져오기"에 노드 이름을 사용하므로, 이름을 알아보기 쉽게 붙이는 습관이 필요합니다.

화면 이동 흐름

```
로그인 → 워크플로우 목록 → 워크플로우 클릭/생성 → Canvas 편집 화면
```

✅ 실습 미니 체크

1. 목록 화면에서 `Create Workflow` 버튼으로 새 워크플로우 만들어보기
2. Canvas에서 `+` 버튼을 눌러 아무 노드나 검색해보기
3. 노드를 하나 배치하고 더블클릭해서 설정창 열어보기
4. 노드 이름을 바꿔보기
5. Canvas 상단의 Editor / Executions 탭 위치 확인하기

1.4 Expression 문법 기초

1.4.1 `{{ $json.필드명 }}` 이해하기

앞 노드의 결과값을 다음 노드에서 쓰려면 **Expression(표현식)** 문법이 필요합니다.

```
{{ $json.필드명 }}
```

- `$json` : 바로 이전 노드에서 넘어온 **현재 아이템**의 데이터
- `.필드명` : 그 데이터(JSON) 안에서 꺼내올 값의 key

예시

이전 노드의 출력이 아래와 같다고 하면,

```
{
  "name": "홍길동",
  "email": "hong@example.com",
  "order_count": 3
}
```

원하는 값	작성할 Expression
이름	`{{ $json.name }}`
이메일	`{{ $json.email }}`
주문 횟수	`{{ $json.order_count }}`

다른 노드의 값을 가져올 때

```
{{ $('노드이름').item.json.필드명 }}
```

바로 앞 노드가 아니라 특정 이름의 노드에서 값을 가져올 때 씁니다. 노드를 여러 개 거치면 앞쪽 노드의 데이터는 `$json`에서 사라지므로, Chapter 2부터 자주 사용합니다.

> ⚠️ **초보자가 자주 하는 실수 세 가지**
> 1. **필드명 오타** — 대소문자를 구분합니다. `Name`과 `name`은 다른 필드입니다.
> 2. **Expression 모드로 전환하지 않음** — 입력창이 Fixed 모드이면 값이 글자 그대로 저장됩니다. n8n 2.x는 `{{ }}`가 포함된 값을 입력하면 자동으로 Expression 모드로 바꾸지만, 값을 고치는 과정에서 Fixed 모드로 남는 경우가 있습니다. 입력창 위에 마우스를 올리면 나타나는 **Fixed / Expression** 표시로 현재 모드를 확인합니다.
> 3. **문자열에 따옴표 누락** — `{{ }}` 안의 텍스트는 `{{ '안녕하세요' }}`처럼 작은따옴표로 감싸야 합니다.

> 💡 Expression 모드로 전환하면 입력창 아래에 **결과 미리보기**가 실시간으로 표시됩니다. 값이 제대로 나오는지 여기서 바로 확인하세요. `[undefined]`가 보이면 필드명이 틀린

> 것입니다.

1.4.2 데이터 흐름 — 아이템 단위 전달

n8n에서 데이터는 항상 **아이템(item)의 배열**로 이동합니다. 처음엔 헷갈리지만 이후 모든 실습의 기초가 되는 개념입니다.

- 하나의 노드는 여러 개의 **아이템**을 출력할 수 있습니다. (공휴일 API가 10개를 반환하면 → 아이템 10개 출력)
- 다음 노드는 기본적으로 **각 아이템마다 한 번씩** 실행됩니다. (엑셀 자동 채우기와 비슷합니다)
- `$json`은 항상 "지금 처리 중인 그 아이템 하나"를 가리킵니다.

```
[공휴일 API 노드]                    [다음 노드]
아이템 1 (공휴일1) ──────────▶ 실행 1회차 ($json = 공휴일1 데이터)
아이템 2 (공휴일2) ──────────▶ 실행 2회차 ($json = 공휴일2 데이터)
아이템 3 (공휴일3) ──────────▶ 실행 3회차 ($json = 공휴일3 데이터)
```

✅ 체크포인트 질문

- 앞 노드에서 이메일 주소 10개가 넘어왔다면, 다음 노드는 몇 번 실행될까요? → **10번**
- `{{ $json.age }}`에서 `age`는 무엇을 의미하나요? → **가져오려는 데이터의 필드(key) 이름**
- HTTP Request 노드 두 개를 연달아 연결했을 때, 두 번째 노드 뒤에서 첫 번째 노드의 결과를 쓰려면? → `$('첫 번째 노드 이름').item.json.필드명`

1.5 실습 1-1. 첫 워크플로우 — 날씨·환율 조회

> 🎯 **목표**: Manual Trigger로 워크플로우를 실행하고, 날씨 API와 환율 API를 차례로 호출한 뒤, 원하는 형태의 문장으로 가공한다.
> ⏱️ **소요 시간 20분 · 사전 준비 없음** (Open-Meteo 날씨, Frankfurter 환율 모두 가입·API Key 불필요)

완성 후 흐름

```
Manual Trigger → 날씨 조회(HTTP Request) → 환율 조회(HTTP Request) → 브리핑 작성(Edit Fields)
```

> 🔗 **1일차 연결**: 1일차 Chapter 4에서 같은 Open-Meteo API를 Python `requests`로 호출했습니다. 그때 쿼리 파라미터(`?latitude=...&longitude=...`)와 `response.json()`으로 했던 일을, 오늘은 HTTP Request 노드 하나가 대신합니다.

1.5.1 Manual Trigger 설정

1. 새 워크플로우 생성 (**Create Workflow**)
2. 좌측 상단의 워크플로우 이름을 클릭해 `실습1_날씨환율조회`로 변경
3. Canvas의 `+` 버튼 클릭 → `Manual Trigger` 검색 후 추가

Manual Trigger는 "실행" 버튼을 눌렀을 때 워크플로우를 시작시키는 가장 기본적인 트리거입니다. 자동 실행이 아니라 **내가 원할 때 돌려보는 용도**라, 개발 중에는 거의 항상 이걸로 시작합니다.

1.5.2 날씨 조회 — HTTP Request

1. Manual Trigger 노드 오른쪽의 `+` 클릭 → `HTTP Request` 노드 검색 후 추가
2. 노드 이름을 `날씨 조회`로 변경 (설정창 상단 이름 클릭)
3. 아래와 같이 입력

항목	값
Method	`GET`
URL	`https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.9780&current_weather=true`

URL이 길어 보이지만 구조는 단순합니다.

파라미터	의미
`latitude=37.5665`	위도 (서울시청 기준)
`longitude=126.9780`	경도
`current_weather=true`	현재 날씨를 함께 달라는 요청

4. 노드의 ▶ **버튼**을 눌러 이 노드까지만 실행
5. 우측 OUTPUT 패널에서 결과 JSON 확인 — `current_weather` 안에 `temperature`, `windspeed` 값이 보이면 성공입니다

> 💡 `windspeed`의 단위는 **km/h**(시속)입니다. m/s(초속)가 아니므로 문장을 만들 때 단위를 정확히 적습니다.

🌍 내 지역으로 바꿔보기

서울이 아닌 다른 지역의 날씨를 보고 싶다면 위도·경도만 바꾸면 됩니다. 좌표는 Google 지도에서 원하는 위치를 우클릭하면 맨 위에 표시됩니다.

지역	latitude	longitude
부산	35.1796	129.0756
대구	35.8714	128.6014
광주	35.1595	126.8526
제주	33.4996	126.5312

> ⚠️ **결과가 비어 있거나 오류가 난다면** URL 오타(특히 `?`와 `&` 위치), Method가 `GET`인지, 위도/경도 값에 공백이 섞이지 않았는지 확인합니다.

1.5.3 환율 조회 — HTTP Request 하나 더

1. `날씨 조회` 노드 오른쪽 `+` 클릭 → `HTTP Request` 추가
2. 노드 이름을 `환율 조회`로 변경
3. 아래와 같이 입력

항목	값
Method	`GET`
URL	`https://api.frankfurter.dev/v1/latest?base=USD&symbols=KRW,JPY`

4. ▶ 버튼으로 실행 → OUTPUT에 아래와 같은 구조가 나오면 성공입니다

```
{
  "amount": 1,
  "base": "USD",
  "date": "2026-10-23",
  "rates": {
    "JPY": 152.4,
    "KRW": 1385.2
  }
}
```

> 💡 Frankfurter는 유럽중앙은행(ECB) 고시 환율을 제공합니다. **영업일 기준 하루 한 번** 갱신되므로 실시간 시세는 아니며, 주말에는 금요일 값이 나옵니다. `date` 필드가 그 기준일입니다. 사내 보고용 참고 환율로는 충분합니다.

1.5.4 브리핑 문장 만들기 — Edit Fields

이제 받아온 숫자를 사람이 읽을 문장으로 바꿉니다. 여기서 중요한 점이 하나 있습니다. `환율 조회` **노드 뒤에서** `$json` **은 환율 데이터만 가리킵니다.** 날씨 데이터는 `$json`에 없으므로, 1.4.1에서 배운 "다른 노드의 값 가져오기"를 사용해야 합니다.

1. `환율 조회` 노드 오른쪽 `+` 클릭 → `Edit Fields (Set)` 노드 추가
2. 노드 이름을 `브리핑 작성`으로 변경
3. `Add Field`를 눌러 아래 세 필드를 추가 (Value는 모두 **fx 아이콘으로 Expression 모드** 전환 후 입력)

Field Name	Type	Value
`weather_message`	String	`{{ '서울 현재 기온은 ' + $('날씨 조회').item.json.current_weather.temperature + '도, 바람은 시속 ' + $('날씨 조회').item.json.current_weather.windspeed + 'km 입니다.' }}`
`rate_message`	String	`{{ '1달러는 ' + $json.rates.KRW + '원, 100엔은 ' + Math.round($json.rates.KRW / $json.rates.JPY * 100) + '원입니다. (' + $json.date + ' 고시)' }}`
`usd_krw`	Number	`{{ $json.rates.KRW }}`

입력하는 동안 아래쪽 미리보기에 완성된 문장이 실시간으로 나타납니다.

> 💡 `rate_message`의 엔화 계산은 "1달러 = KRW원이고 1달러 = JPY엔이므로, 1엔 = KRW/JPY원"이라는 환산입니다. `Math.round()`는 소수점을 반올림하는 JavaScript 함수로, Expression 안에서 그대로 사용할 수 있습니다. `usd_krw`는 Chapter 2에서 조건 분기에 사용하기 위해 숫자 그대로 따로 담아둡니다.

4. ▶ 버튼으로 실행 → OUTPUT에 세 값이 모두 나오는지 확인

🎉 완성 확인

OUTPUT 패널에 아래와 같이 표시되면 실습 성공입니다. (숫자는 실행 시점에 따라 다릅니다)

```
{
  "weather_message": "서울 현재 기온은 14.2도, 바람은 시속 7.6km 입니다.",
  "rate_message": "1달러는 1385.2원, 100엔은 909원입니다. (2026-10-23 고시)",
```

```
  "usd_krw": 1385.2
}
```

마지막으로 우측 상단 **Save** 버튼을 눌러 워크플로우를 저장합니다. **이 워크플로우는 Chapter 2에서 그대로 이어서 사용하므로 반드시 저장해두세요.**

📌 Chapter 1 요약

개념	핵심 내용
AI Agent	목표만 주면 **도구 선택까지** LLM이 스스로 판단하는 것
n8n 설치	Node.js 24 → `npx n8n@2` → `localhost:5678` 접속, 터미널은 켜둔 채 유지
화면 구조	워크플로우 목록 → Canvas(작업공간) + Node(작업단위) + Execution 로그(실행기록)
Expression	`{{ $json.필드명 }}`으로 이전 노드 데이터 참조, **fx 아이콘으로 모드 전환 필수**
다른 노드 참조	`$('노드이름').item.json.필드명` — 노드를 거치면 앞 데이터는 `$json`에서 사라짐
데이터 흐름	데이터는 "아이템의 배열"로 이동하며, 다음 노드는 아이템 개수만큼 반복 실행
실습 1-1	Manual Trigger → 날씨 조회 → 환율 조회 → 브리핑 작성(문장 2개 + 숫자 1개)

다음 Chapter 예고: Chapter 2에서는 Trigger의 종류(Webhook, Schedule, Chat)를 배우고, 방금 만든 워크플로우를 **매일 아침 8시에 자동으로 발송되는 브리핑**으로 바꿉니다. 환율이 기준을 넘으면 경고 문구가 붙도록 조건 분기(IF, Switch)도 추가합니다.